# Fast exact overlap minimization for three-dimensional irregular strip packing of polyhedra

Ignacio Araya[a,*]

[a]*Escuela de Ingeniería Informática, Pontificia Universidad Católica de Valparaíso, Chile*

**Abstract**

We address the three-dimensional irregular strip packing problem: polyhedra, possibly non-convex, must be placed by translation in a container with a fixed rectangular base so that the height is minimized. We propose an overlap-minimization heuristic that works with the exact geometry of the pieces, represented as unions of convex parts. Its new element is an exact axis move. The overlap of two parts is measured by the penetration depth of their relative position in their no-fit polytope; along any axis this function is zero outside an interval and concave inside it, so the best position of a piece along an axis is found by evaluating a few breakpoints only. The rest of the method adapts a few known components from two-dimensional nesting: a greedy construction, a strategic oscillation of the container height with guided local search weights, a local improvement and restarts. On the benchmarks of polyhedral pieces, the method improves the best known heights of 11 of 13 instances within two minutes, reaches in 10 seconds the quality that the previous exact local search attains in 250 seconds (time normalized to our hardware), and matches 24 of the 27 known optima of small instances. An ablation study shows that the speed of the move, the guided local search weights and the effort spent in each separation are the decisive elements.

*Keywords:* Cutting, Packing, 3D irregular packing, No-fit polytope, Guided local search, Linear programming

*Corresponding author

*Email address:* ignacio.araya@pucv.cl (Ignacio Araya)

## 1. Introduction

Three-dimensional irregular packing arises in additive manufacturing (packing parts in the build volume of a printer), in the loading of non-rectangular goods, and in the design of compact assemblies. In its open-dimension variant, the *3D irregular strip packing problem*, a set of polyhedra must be placed without overlap inside a container with a fixed $W \times D$ base, and the height of the packing is to be minimized. We consider the translational version: pieces keep their orientation.

The literature on this problem is small compared with its two-dimensional counterpart, and it is split by the representation of the geometry. Exact approaches use the polyhedra directly: phi-functions and nonlinear programming (Stoyan et al., 2004, 2005; Romanova et al., 2018), the 3DNest local search of Egeblad et al. (2009), which minimizes the exact overlap volume along axis-parallel translations, constructive heuristics such as HAPE3D (Liu et al., 2015), and, recently, mixed-integer models based on no-fit polyhedra that solve small convex instances to optimality (Tollenaere et al., 2026). Approximate approaches discretize the pieces into voxels (Lamas-Fernandez et al., 2023); voxels make overlap tests trivial and scale well, but they overestimate the pieces and the quality of the solutions depends on the resolution.

In two dimensions, the most successful methods for irregular strip packing are overlap-minimization heuristics: the height (length) is decreased, the pieces are allowed to overlap, and a local search moves them until the overlap vanishes (Egeblad et al., 2007; Umetani et al., 2009; Imamichi et al., 2009; Elkeran, 2013; Gardeyn et al., 2025). The key ingredients are a cheap and informative overlap measure, moves that can be optimized exactly or quickly, and guided local search (GLS) weights to escape local minima. This paper brings these ingredients to three-dimensional polyhedra with an exact representation. Most components of the method are adaptations of known ideas, whose origin is given in Section 3. The new element is the overlap measure and the move built on it: the penetration depth in the no-fit polytopes (NFPs) of convex parts which, unlike the overlap volume used by 3DNest, can be minimized exactly along an axis by evaluating a few breakpoints.

*Contributions.*

1. A fast exact overlap-minimization move for 3D polyhedra represented as unions of convex parts. It brings the exact axis moves of Umetani et al. (2009), defined for 2D polygons with directional penetration

depths, to 3D and non-convex pieces: the overlap is measured by the penetration depth in the NFP of each pair of convex parts, which along an axis is concave on an interval and zero outside it, so the optimal position of a piece along an axis is attained at an interval endpoint (Proposition 1). With an implementation that evaluates each term only at the breakpoints inside its interval, the move is much cheaper than the exact overlap volume of Egeblad et al. (2009), and this speed is what makes exact geometry practical in a 3D metaheuristic.

2. A method built around this move with few components, adapted from known methods (a bottom-left construction with gravity, a strategic oscillation of the height, multiplicative GLS weights (Gardeyn et al., 2025), a compaction of the feasible layouts and restarts), and an ablation study showing which elements matter: the GLS weights, the speed of the move and the effort spent in each separation are decisive, the compaction is needed to reach the optima, and the perturbation of the restarts and the tuned construction criterion have no significant effect.
3. New best known solutions on the Merged instances of Egeblad et al. (2009) and on classical instances of Stoyan et al. (2004, 2005), with a time-matched comparison against the exact method 3DNest, and a comparison with the optimal values and bounds of the mixed-integer models of Tollenaere et al. (2026).
4. An exact convex decomposition of the non-convex (prismatic) pieces of Ikonen et al. (1997) used in the Merged instances.

Section 2 defines the problem and the geometric tools. Section 3 describes the algorithm, and Section 4 the experiments.

## 2. Problem and geometry

### *2.1. Problem*

We are given a container base of width $W > 0$ and depth $D > 0$, of unbounded height, and $n$ pieces $P_1, \ldots, P_n \subset \mathbb{R}^3$ (polyhedra, with multiplicities). Each piece is a union of convex polytopes, $P_i = \bigcup_{a \in \mathcal{A}_i} A_a$, expressed in a local frame where the minimum of its bounding box is the origin; $w_i, d_i, h_i$ denote its bounding-box dimensions. A solution is a vector of translations $t_i = (x_i, y_i, z_i) \in \mathbb{R}^3$, $i = 1, \ldots, n$ (continuous positions; rotations are not

allowed), such that

$$0 \le x_i \le W - w_i, \quad 0 \le y_i \le D - d_i, \quad z_i \ge 0, \qquad i = 1, \dots, n, \tag{1}$$

$$\operatorname{int}(P_i + t_i) \cap \operatorname{int}(P_j + t_j) = \emptyset, \qquad i < j, \tag{2}$$

where $\operatorname{int}(\cdot)$ denotes the interior of a set (pieces may touch but not overlap), and the objective is to minimize $H = \max_i(z_i + h_i)$. A trivial lower bound is $H_0 = \max\{\max_i h_i,\ \sum_i \operatorname{vol}(P_i)/(WD)\}$.

*2.2. No-fit polytopes and penetration depth*

For two convex parts $A$ and $B$ placed at $t_A$ and $t_B$, their interiors intersect if and only if the relative position $r = t_A - t_B$ lies in the interior of the no-fit polytope

$$N_{AB} = B \oplus (-A) = \operatorname{conv}\{b - a :\ a \in V(A),\ b \in V(B)\}, \tag{3}$$

where $V(\cdot)$ is the vertex set (Bennell and Oliveira, 2008). We compute $N_{AB}$ once per pair of parts as the convex hull of the vertex differences and store it in H-representation, $N_{AB} = \{r :\ n_f \cdot r \le d_f,\ f \in F_{AB}\}$, with unit normals $\|n_f\| = 1$.

**Definition 1.** *The* penetration *of $A$ and $B$ at relative position $r$ is* $\delta_{AB}(r) = \max\{0,\ \min_{f \in F_{AB}}(d_f - n_f \cdot r)\}$.

For $r$ inside $N_{AB}$, $\delta_{AB}(r)$ is the Euclidean distance from $r$ to the boundary of $N_{AB}$, i.e., the length of the shortest translation that separates $A$ and $B$, known as the translational penetration depth (Cameron and Culley, 1986; Agarwal et al., 2000). It is zero if and only if the parts do not overlap. For two pieces we use $\delta_{ij}(t_i - t_j) = \sum_{a \in \mathcal{A}_i} \sum_{b \in \mathcal{A}_j} \delta_{ab}(t_i - t_j)$, where only pairs of parts whose bounding boxes intersect are evaluated. Figure 1 illustrates both definitions in two dimensions.

*2.3. Exact minimization along an axis*

Let all pieces but $i$ be fixed, and let $i$ move along axis $e_k$, $t_i(s) = t_i + (s - t_{ik})e_k$, with $s \in [0, U_k]$ (the container limits, using the current height $H$ for the vertical axis). The weighted overlap of $i$ is

$$\phi(s) = \sum_{j \ne i} w_{ij}\, \delta_{ij}\big(t_i(s) - t_j\big), \tag{4}$$

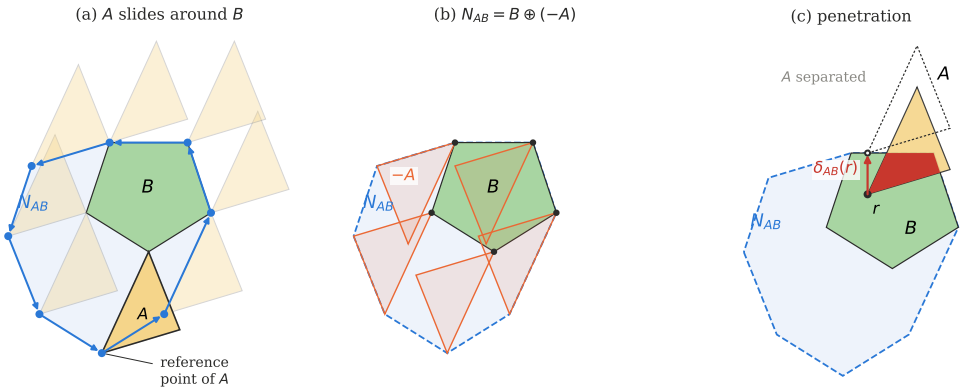



Figure 1: No-fit polytope and penetration in 2D. (a) When $A$ slides around $B$ keeping contact, its reference point (the minimum of its bounding box) traces the boundary of $N_{AB}$; the shaded copies of $A$ touch $B$ at the vertices of $N_{AB}$. (b) Equivalently, $N_{AB} = B \oplus (-A)$ is the convex hull of the copies of $-A$ placed at the vertices of $B$. (c) If $r$ lies inside $N_{AB}$ the parts overlap (red), and $\delta_{AB}(r)$ is the distance from $r$ to the nearest facet: the shortest translation of $A$ that separates the parts (dotted).

where the weights $w_{ij} \geq 1$ (initially 1) are the penalties of guided local search (GLS) (Voudouris and Tsang, 1999): they are increased for the pairs that remain overlapping at local minima, so that the search focuses on them (Section 3.2).

**Proposition 1.** *Each term $\delta_{ab}(t_i(s) - t_j)$ is zero outside an interval $(L_{ab}, U_{ab})$ and concave inside it. Hence $\phi$ is concave on every interval between consecutive points of $\mathcal{B} = \{0, U_k\} \cup \{L_{ab}, U_{ab}\}$, and a minimizer of $\phi$ on $[0, U_k]$ is attained at a point of $\mathcal{B}$.*

*Proof.* Along the line, $d_f - n_f \cdot r(s) = c_f - n_{fk} s$ is affine. The set where all of them are positive is the open interval $(L_{ab}, U_{ab})$ with $L_{ab} = \max_{n_{fk}<0} c_f/n_{fk}$ and $U_{ab} = \min_{n_{fk}>0} c_f/n_{fk}$ (facets with $n_{fk} = 0$ only decide whether the interval is empty). Inside it, $\delta_{ab}$ is a minimum of affine functions, hence concave; outside it is zero. Between consecutive points of $\mathcal{B}$ every term is either concave or identically zero, so $\phi$ is concave there and attains its minimum at an endpoint. □

Proposition 1 is the counterpart, for the Euclidean penetration depth of convex parts in 3D, of the piecewise linear directional penetration depths of Umetani et al. (2009) in 2D, which require a trapezoidal decomposition of each NFP and have no practical equivalent for 3D NFPs. With pieces

decomposed into convex parts, every NFP is convex and each term reduces to a minimum of affine functions. Figure 2 illustrates the proposition in two dimensions.

The move evaluates $\phi$ at the points of $\mathcal{B}$ only. With the interval of each term known, a term is evaluated only at the candidates inside it: sorting the candidates, each term contributes to a contiguous range found by binary search. Facets are stored as $(c_f, n_{fk})$ in contiguous arrays, so evaluating a term costs one fused multiply-add per facet. Ties are broken towards the smaller coordinate (gravity and walls).

## 3. Algorithm

Algorithm 1 summarizes the method. A greedy construction gives a first feasible solution $S$ of height $H$. The search then tries to fit the pieces in a slightly lower container of height $h < H$: the positions of $S$ are *clamped* to $h$, which creates overlaps, and MinimizeOverlap moves the pieces to remove them. Each call of MinimizeOverlap is a *separation attempt*. It *succeeds* if it reaches a layout without overlap, which then fits in height $h$; it *fails* if overlap remains after its maximum number of passes. After a success, the layout is compacted and becomes the new best solution, and the target is lowered again. After a failure, the target is raised slightly and the attempt continues from the overlapping layout; when the target reaches $H$, the search restarts from the best solution. The components are described in the order in which they are needed: the construction (Section 3.1), the overlap minimization (Section 3.2), the oscillation of the target height and the restarts (Section 3.3), and the compaction of the feasible layouts (Section 3.4).

### *3.1. Greedy construction*

Pieces are placed one at a time on top of the current load (Algorithm 2). For each type of piece still to be placed, $g \times g$ starting points ($g = 4$) are taken on a regular grid of the base, at the current height $H_{\text{cur}}$ of the load, and from each of them the piece descends (Descend, lines 9–18). The descent is a sequence of classical linear programs (LPs) in the three translation variables $u = (x, y, z)$ of the piece. At the current point $t$, for every placed part $b$ that the part $a$ of the piece may meet within a trust region (a box of half side $\Delta$, the largest dimension of the piece), one facet $n_f \cdot r \geq d_f$ of $N_{ab}$ satisfied at $t$ is selected: the one that allows the longest step in the descent direction (line 13). Since $N_{ab}$ is convex, the half-space beyond any of its

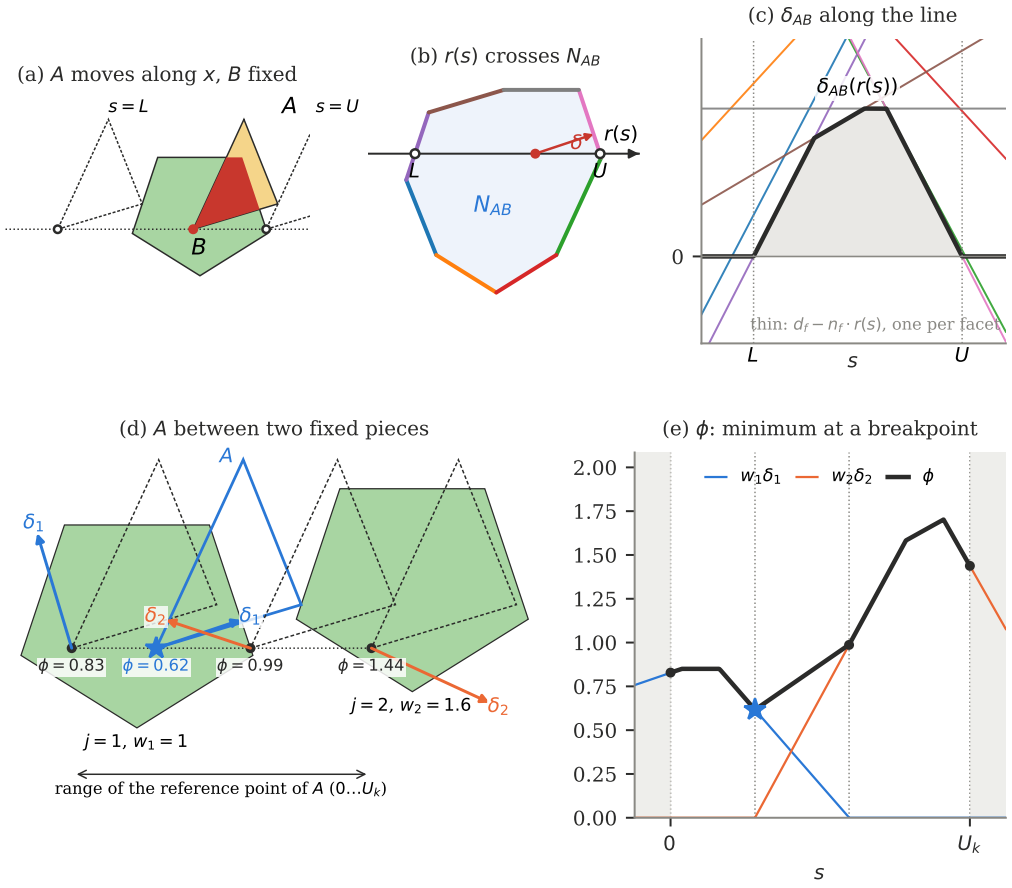



Figure 2: Proposition 1 in 2D, with the parts of Figure 1. (a) Part $A$ moves along $x$ with $B$ fixed: it touches $B$ at $s = L$ and $s = U$ (dashed) and overlaps it in between (red). (b) In the space of relative positions, $r(s)$ moves along a line that enters $N_{AB}$ at $L$ and leaves it at $U$. (c) For each facet (same colors as in (b)), $d_f - n_f \cdot r(s)$ is affine in $s$; the penetration is the lower envelope of these lines clipped at zero: zero outside $(L, U)$ and concave inside. (d)–(e) $A$ moves between two fixed pieces within the container range $[0, U_k]$. In (d), $A$ is drawn at each breakpoint of (e) ($0$, $U_k$ and the $L$, $U$ of each term; dots: its reference point), with arrows for its penetration with each piece it overlaps ($\delta_1$, $\delta_2$) and the resulting $\phi = w_1\delta_1 + w_2\delta_2$ (weights $w_1 = 1$, $w_2 = 1.6$): the two central positions have similar penetrations but different $\phi$. Between consecutive breakpoints the weighted overlap $\phi$ is a sum of concave functions, so its minimum (star, solid outline in (d)) is at a breakpoint, and only these points are evaluated.

**Algorithm 1:** Overview

```
1  S ← Compact(Greedy()); H ← H(S);
2  h ← (1 − ρ)H; T ← Clamp(S, h); w ← 1;            // overlaps appear
3  while time remains do
4      if MinimizeOverlap(T, h, w) then      // success: T is feasible
5          S ← Compact(T); H ← H(S);  // new best solution, H ≤ h
6          h ← (1 − ρ)H; T ← Clamp(S, h); w ← 1;
7      else                                                  // failure
8          h ← h + ρH/4; T ← Clamp(T, h);  // keeps the overlaps and
             w
9          if h ≥ H then
10             h ← (1 − ρ)H; T ← Clamp(S, h); move a random piece
                 of T; w ← 1;                                  // restart
11 return S;
```

facets contains no overlapping position, so the selected facets define a convex region of positions where the piece does not overlap the placed parts. The LP of line 14 is

$$\begin{aligned} \min_{u} \quad & c \cdot u \\ \text{s.t.} \quad & n_f \cdot (u - t_b) \geq d_f \quad \text{for every selected facet } f \text{ of } N_{ab}, \\ & \max(\ell,\, t - \Delta) \leq u \leq \min(\bar{u},\, t + \Delta), \quad u_z \leq t_z, \end{aligned} \tag{5}$$

where $t_b$ is the position of the piece of $b$ and $[\ell, \bar{u}]$ are the positions that keep the piece inside the container. The objective $c = (\pm\sigma, \pm\sigma, 1)$ moves the piece down and, with a small weight $\sigma$, towards the walls in $x$ and $y$ nearest to the starting point (the signs are fixed at the start, so the objective is linear). The current point $t$ is feasible, and every feasible point is a position without overlap, so each LP can only improve the position. With three variables and a few dozen constraints, the LP is solved by a small primal active-set method (a simplex method in the space of $u$) that starts at $t$ and keeps every iterate feasible; no external solver is needed. The facets are re-selected at the new point and the procedure stops when the objective no longer decreases. Figure 4 shows the LPs of one descent. Descend combines two classical ideas: bottom-left placement rules for irregular pieces (Jakobs, 1996), here with priority to gravity, as the minimum potential energy

principle of HAPE3D (Liu et al., 2015), and the compaction of a layout by linear programs over NFP facets (Li and Milenkovic, 1995; Gomes and Oliveira, 2006), here applied to a single piece.

Among all piece types and resulting positions, the greedy places the pair $(k, t)$ that maximizes (line 6)

$$\begin{aligned} val(k,t) = \alpha \log \frac{V_k}{V_{\max}} - \beta \frac{z}{H_0} \\ - \gamma \frac{\max(0,\, z + h_k - H_{\text{cur}})}{H_0}, \end{aligned} \tag{6}$$

where $V_k$ is the volume of the piece, $V_{\max}$ the largest volume, $z$ the height of the position, $h_k$ the height of the piece and $H_{\text{cur}}$ the current height of the load. The first term favors large pieces, as the classical orderings by decreasing volume, but on a logarithmic scale, so that a smaller piece is preferred when it fits clearly lower. The second term favors low positions, i.e., filling the gaps of the load before building on top of it, and the third penalizes positions that raise the height of the load. Normalizing by the lower bound $H_0$ makes the parameters independent of the scale of the instance. For a given piece the score decreases with $z$, so the lowest of its positions is always preferred; the score thus decides which piece is placed next: a large piece if it fits in a low gap, and a smaller one if the large pieces would raise the height (the values of the parameters are discussed in Section 4). Figure 3 illustrates the construction in two dimensions.

### *3.2. Overlap minimization*

Given a layout $T$ with overlaps in a container of height $h$, MinimizeOverlap minimizes the total overlap $\sum_{i<j} \delta_{ij}$ by moving one piece at a time along an axis. Moving one piece at a time along the axes to the position of minimum overlap is the neighborhood of Egeblad et al. (2007, 2009), who minimize the overlap area or volume; Umetani et al. (2009) and Imamichi et al. (2009) measure the overlap of 2D polygons by the penetration depth in the NFP, which we use for 3D unions of convex parts.

Such a local search soon reaches local minima, layouts in which no single move reduces the overlap. To escape them we use guided local search (GLS) (Voudouris and Tsang, 1999), as Egeblad et al. (2007) and Umetani et al. (2009) do for nesting. GLS changes the objective instead of the layout: each pair of pieces has a weight $w_{ij} \geq 1$, the moves minimize the weighted overlap

**Algorithm 2:** Greedy construction

```
function GREEDY()
    L ← ∅ (placed pieces); H_cur ← 0;
    while some piece is not placed do
        foreach type k with pieces left and each of g × g grid points (x, y) of the base do
            t ← DESCEND(k, (x, y, H_cur), L);
            val ← val(k, t) of Eq. (6);
        place the pair (k, t) of largest val in L;
        H_cur ← max(H_cur, z + h_k);
    return L;
function DESCEND(k, t, L)
    c ← (±σ, ±σ, 1);                    // down and towards the nearest walls
    repeat
        foreach part a of k and placed part b of L that may meet a in the trust region do
            select the facet f of N_ab satisfied at t that allows the longest step along −c;
        t′ ← optimal solution of the LP (5) built at t;
        if c · t′ ≥ c · t then break;
        t ← t′;
    until no improvement;
    return t;
```

$\phi = \sum_{i<j} w_{ij}\,\delta_{ij}$ of Eq. (4), and at a local minimum the weights of the pairs that still overlap are increased. A persistent overlap thus becomes more expensive, and the next moves remove it even at the cost of creating smaller overlaps elsewhere, which in turn gain weight if they persist. We update the weights multiplicatively as in Gardeyn et al. (2025): at each local minimum, the weight of every overlapping pair is multiplied by $1.2 + 0.8\,\delta_{ij}/\max\delta$, and the weights of the other pairs decay by a factor 0.95 (minimum 1). All the weights start at 1. Each element of this rule has a simple purpose:

- *Multiplicative growth.* The weight of a pair that keeps overlapping grows geometrically: after five consecutive local minima it is between

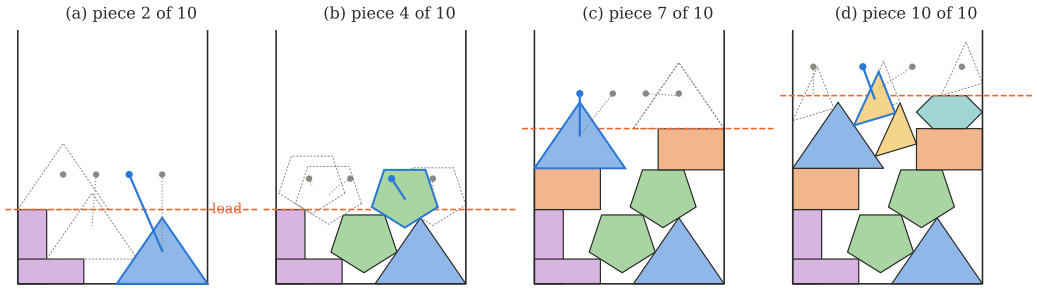



Figure 3: Greedy construction on a 2D example (10 pieces, one of them a union of two convex parts), computed with a 2D version of the method. In each step, a piece starts from $g = 4$ points on top of the current load (dots) and descends by a sequence of LPs over NFP facets, towards the floor and the nearest wall; dotted outlines: final positions of the other starts; blue: descent and position chosen by the greedy criterion.

$1.2^5 \approx 2.5$ and $2^5 = 32$ times its initial value. A persistent overlap therefore soon dominates $\phi$, however many other pairs there are, and the search is forced to remove it. With additive weights ($w_{ij} \leftarrow w_{ij}+1$) the growth is linear, so the relative pressure on a persistent pair weakens as the weights of all pairs accumulate.

- *Factor proportional to the penetration.* The factor ranges from 1.2, for the shallowest overlap, to 2, for the deepest one. Shallow overlaps are often removed by the next moves without help, while deep ones need a large change of the layout, so the deepest overlaps are penalized fastest.

- *Decay.* The weight of a pair that no longer overlaps returns slowly to 1. The weights thus reflect the recent history of the search: a pair that overlapped long ago does not keep constraining the moves, and its weight starts growing from a low value if it overlaps again.

The ablation of Section 4 confirms that this rule matters: additive weights increase the heights by 1.4% (Table 2).

In Algorithm 3, line 2 computes the penetrations of all pairs, and each iteration of the loop is a pass. Only the pieces involved in an overlapping pair are moved (line 7), in random order, each along the three axes in random order with the exact move of Proposition 1 (line 9): $\phi$ is evaluated at the breakpoints and the best one is taken; since the current position is a can-

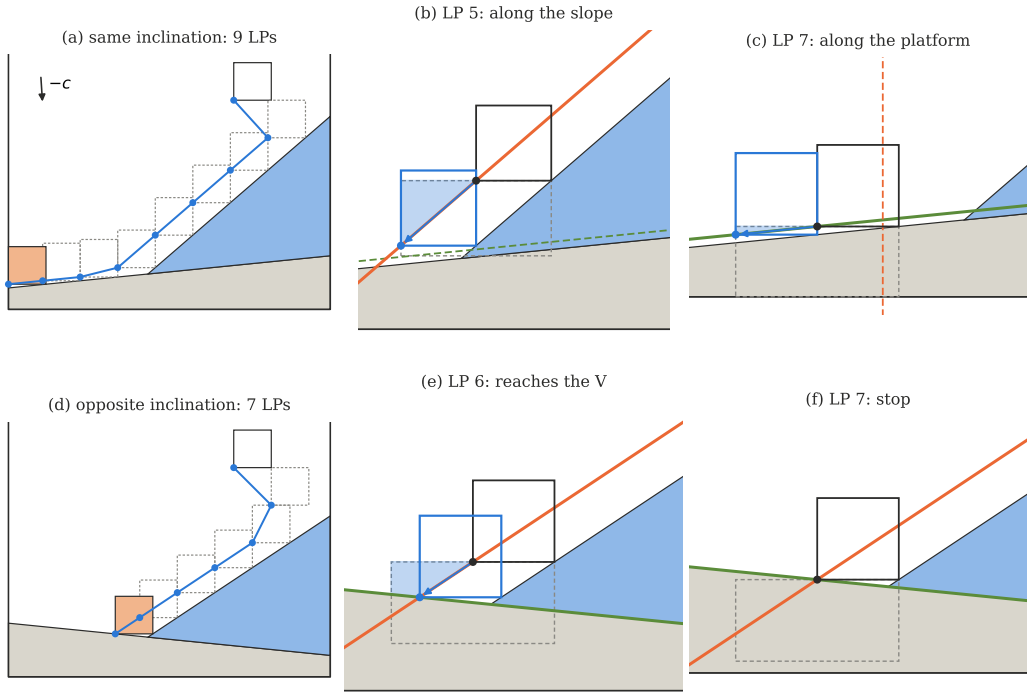



Figure 4: DESCEND step by step on a 2D example: a square falls onto a slope that ends on a platform whose top is slightly inclined, (a)–(c) in the same direction as the slope, (d)–(f) in the opposite direction. (a), (d) The whole descent: each dot is the reference point of the square (the corner of its bounding box) after one LP, and the dotted boxes are the trust regions. In free space the optimum of the LP is a corner of the trust region, so the square moves down and towards the nearest wall; once it touches the slope it slides along it. (b), (c), (e), (f) Single LPs (Algorithm 2, line 14) in the space of the reference point: $t$ is the current point, the dashed box the trust region (the point may not go up), each line the facet selected for a placed part (orange: slope; green: platform; solid if active at the optimum), the shaded region the feasible region, and the arrow the move to the optimum along $-c$. (b) The square slides along the slope; the facet of the platform is already selected but not active yet. (c) On the platform, the facet of the platform is active and the square keeps sliding, down the slight incline, to the wall. (e) With the opposite inclination, the square reaches the V between the slope and the platform. (f) Both facets are active and no direction decreases $c \cdot t$: the descent stops. Gravity has priority over the wall term, so the piece slides down any incline.

didate, a move never increases $\phi$. After moving a piece, only its row of the penetration matrix is recomputed (line 11). A pass without gain (line 12) is a local minimum of $\phi$, and the weights are updated. The procedure succeeds when no overlap remains (then verified with a separating-axis test) and fails after 500 passes. Figure 5 shows all the passes on the 2D example, from a clamped layout.

**Algorithm 3:** Overlap minimization

```
1  function MinimizeOverlap(T, h, w)
2      P_ij ← δ_ij(t_i − t_j) for all pairs;                    // penetration matrix
3      for pass = 1, ..., 500 do
4          O ← {(i, j) : P_ij > 0};
5          if O = ∅ then return true if the separating-axis test
             confirms it;
6          gain ← 0;
7          foreach piece i of a pair of O, in random order do
8              foreach axis k ∈ {x, y, z} in random order do
9                  s* ← arg min_{s∈B} φ(s) over the breakpoints B of
                     Proposition 1, with the container
                     [0, W − w_i] × [0, D − d_i] × [0, h − h_i];
10                 gain ← gain + φ(t_ik) − φ(s*); t_ik ← s*;
11             recompute row i of P;
12         if gain = 0 then                                     // local minimum: GLS
13             w_ij ← max(1, 0.95 w_ij) for (i, j) ∉ O;
               w_ij ← w_ij (1.2 + 0.8 P_ij / max_O P) for (i, j) ∈ O;
14     return false;
```

### *3.3. Strategic oscillation and restarts*

The target height $h$ is set from the best height $H$ and clamped layouts,

$$\textsc{Clamp}(T, h): \quad z_i \leftarrow \min\{z_i,\ h - h_i\}, \qquad i = 1, \ldots, n, \tag{7}$$

which moves every piece that exceeds $h$ down until its top is at $h$, regardless of the other pieces, and thus creates overlaps among the pieces near the top. After a success, the target is lowered to $(1 - \rho)H$, with $\rho = 1\%$, below

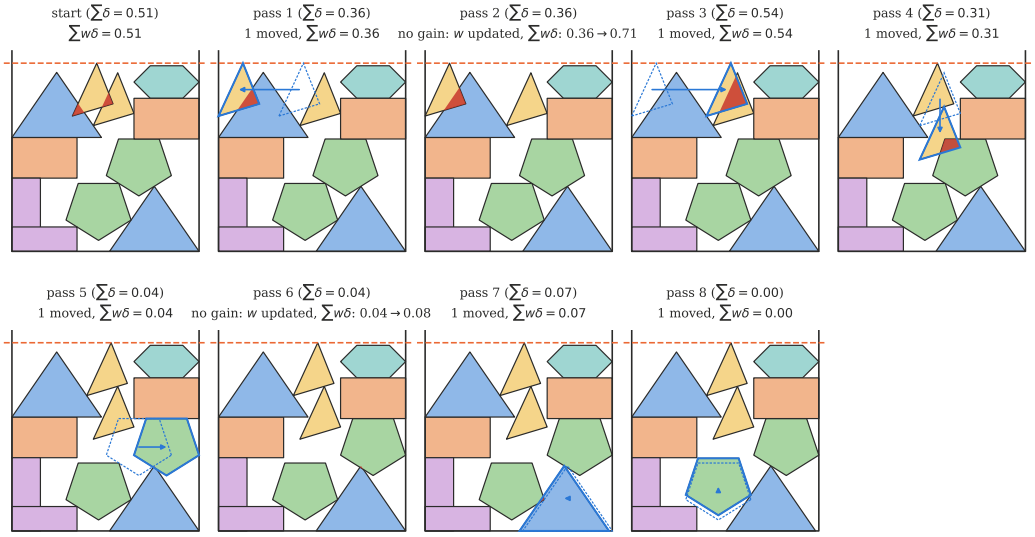



Figure 5: MinimizeOverlap on the 2D example: the greedy layout of Figure 3 is clamped to the target height $h = 0.9H$ (dashed line), and every pass is shown. In each pass, the pieces that overlap are moved along the axes; the pieces actually moved are outlined in blue (dashed: position before the pass), and overlaps are in red. Each title gives the unweighted overlap $\sum \delta$ and the weighted overlap $\sum w\delta$, which is the one minimized: no move increases it. Pass 2 does not reduce it (a local minimum), so the weight of the overlapping pair of triangles is increased from 1 to 2 and $\sum w\delta$ jumps from 0.36 to 0.71; the weights change the objective, not the layout, and the next pass starts from the same positions. Pass 3 then moves the small triangle onto the other one: $\sum w\delta$ decreases to 0.54, although $\sum \delta$ increases. The triangle is moved down in pass 4. The same happens with the pentagon and the triangle at the bottom in passes 6–8, and after pass 8 the layout is feasible.

the new best height, the best solution is clamped to it and the weights are reset. After a failure, the target is raised by $\rho H/4$ and the next attempt starts from the overlapping layout left by the failed one, with its weights, so that a few failures are allowed before giving up the current layout. This scheme of shrinking the container, allowing overlap and separating the pieces comes from the 2D methods of Egeblad et al. (2007), Umetani et al. (2009) and Imamichi et al. (2009), and was used in 3D by Egeblad et al. (2009); the clamp is the counterpart of moving the pieces that exceed the new strip length inside it.

When the target reaches the best height again, the search restarts from the best solution clamped to $(1 - \rho)H$, with the weights reset, after moving one random piece (a kick) to a random position in the container. This is the perturbation step of iterated local search (Lourenço et al., 2003), which Imamichi et al. (2009) applied to nesting. The kick matters on small instances: with two pieces there is a single pair, so the weights have no effect, and without it the search keeps returning to the layout produced by the greedy order (Section 4.4).

### *3.4. Compaction of feasible layouts*

MinimizeOverlap stops as soon as no overlap remains: the layout fits in the target height, but its pieces are where the last moves left them, often with free space below them, since a move along $z$ goes to the position of least overlap, not to the lowest one. Without a further step, the height of the best solution would decrease only in the discrete steps $\rho$ of the oscillation. Compact lets the pieces settle onto each other, in the sense of the compaction of Li and Milenkovic (1995). It is applied to every feasible layout: after each successful separation and after the construction, where it rarely changes anything, since the greedy already leaves each piece at the end of a descent.

A pass of Compact takes the pieces in increasing order of $z$; each one descends again with Descend, with the other pieces fixed, and if it defines the height, it is moved to the lowest of the positions generated as in the greedy construction, if that is lower. Passes are repeated while the height decreases (at most 50). A pass can enable moves that were not possible in the previous one, because the constraints of Descend, the NFP facets of the neighboring pieces, change when those pieces move, and with irregular pieces the order by the bottom is not the order in which they support each other. For example, a piece $A$ may lean on the inclined side of a wedge $B$

whose narrow bottom is higher than the bottom of $A$: $A$ is processed first and cannot move, then $B$ descends into a gap, and only in the next pass can $A$ follow it. Likewise, once the pieces that defined the height have moved lower, other pieces define it and are tried in the next pass. In a run on Stoyan 2004 Example 2, after a separation at height 40.19, the successive passes lowered the height to 39.80, 38.71 and 38.24, by descents alone: most of the gain came after the first pass. With COMPACT, the optima of the small instances are reached exactly (Section 4.4).

## 4. Computational experiments

### *4.1. Setting*

The method is implemented in C++ and needs no external solver. Experiments ran on two Intel Xeon E5-2620 v4 processors (2.1 GHz, 3.0 GHz turbo, 16 cores, 64 GB, Ubuntu 24.04, g++ 13.3), one thread per run, 14 runs in parallel. Each configuration was run with 10 seeds and a time limit of 120 s per instance. All reported solutions were verified with an independent checker that tests every pair of convex parts for a separating axis, with a tolerance of $10^{-6}$ on the penetration depth (the containers are 10 to 60 units wide). Table 1 lists the parameters of the method. Those of the construction were tuned with Optuna (Akiba et al., 2019) on 16 training instances, with an earlier version of the method; the weight update follows Gardeyn et al. (2025); the step $\rho$ and the number of passes are analyzed in Table 2.

*Instances.* We use the 3D instances of the ESICUP repository (EURO Special Interest Group on Cutting and Packing, 2025):

- *Stoyan 2005*, Examples 1–3 (7, 12 and 25 convex pieces) (Stoyan et al., 2005). In the public data, one piece (polytope7) has faces that use only 9 of its 10 vertices; we use its convex hull (volume 192.5), which matches the volume reported by Egeblad et al. (2009).

- *Stoyan 2004*, Examples 1–5 (20–50 non-convex pieces) (Stoyan et al., 2004), each defined as a union of convex polytopes, which we use directly as the decomposition.

- *Merged1–5* (Egeblad et al., 2009): $i$ copies of 15 pieces, the 7 convex pieces of Stoyan 2005 and the 8 pieces of Ikonen et al. (1997) scaled by 4, in a container whose base is such that a layout with 50% utilization

Table 1: Parameters of the method.

| Parameter | Value | Role | Origin |
|---|---|---|---|
| $g$ | 4 | starting points per axis of the base | tuned |
| $\alpha, \beta, \gamma$ | 1, 5.53, 0.77 | greedy criterion (6) | tuned |
| $\sigma$ | 0.065 | wall term of DESCEND | tuned |
| $\Delta$ | largest dimension of the piece | trust region of DESCEND | design |
| $\rho$ | 1% (raised by $\rho/4$) | step of the oscillation | Table 2 |
| passes | 500 | limit of MINIMIZEOVERLAP | Table 2 |
| GLS | $\times(1.2 + 0.8\,\delta_{ij}/\max\delta)$, decay 0.95 | weight update at local minima | Gardeyn et al. (2025) |
| COMPACT | at most 50 passes | compaction of feasible layouts | safeguard |
| kick | one random piece, random position | restart | Table 2 |

is a cube. Six Ikonen pieces are non-convex prisms; we decompose them exactly by triangulating the base polygon, merging triangles into convex polygons (Hertel and Mehlhorn, 1983), and extruding (a frame is split into four boxes; two pieces have polygons touching at a vertex). The volumes of the parts match those of the meshes.

- *Liu Experiment 1* (Liu et al., 2015): 36 pieces (8 tetrahedra, 8 cubes, 8 stars, 8 arrows and 4 rings) in a $20 \times 20$ container; the stars, arrows and rings are prisms and are decomposed exactly in the same way.

*References.* 3DNest (Egeblad et al., 2009) uses the same exact geometry; its average and maximum utilization over 10 runs of 600 s (one core of a 2.16 GHz Core Duo) are converted to heights. Lamas-Fernandez et al. (2023) report best and average heights of a voxel VNS; their voxelization overestimates the pieces (by 15% in volume for Merged1, as they note), so their solutions are feasible for the original pieces. For Stoyan 2004 the only exact reference is Stoyan et al. (2004).

### *4.2. Component analysis*

Table 2 evaluates the components on the seven hardest instances (Stoyan 2004 Examples 1–5 and Stoyan 2005 Examples 2–3). Each row changes one element of the method:

- *Without* COMPACT: no compaction after the construction and the separations (Section 3.4).
- $\sigma = 0$: DESCEND without the wall term (gravity only).
- *Greedy by volume*: the largest piece first, at its lowest position ($\alpha$ large, $\gamma = 0$), instead of the tuned criterion (6).
- *No kick*: the restarts do not move any piece.
- *Additive weights*: $w_{ij} \leftarrow w_{ij} + 1$ for the overlapping pairs at local minima.
- *Time limit of 24 s*: the same runs stopped at 24 s instead of 120 s, i.e., with a fifth of the time.
- *Sensitivity*: step $\rho$ of the oscillation and maximum number of passes of MINIMIZEOVERLAP.

For each variant, the last column of the table gives the mean relative change of the height with respect to the proposed method over the 70 runs (7 instances, 10 seeds), positive when the heights are higher (worse), and the $p$-value of a Wilcoxon signed-rank test that pairs the runs by instance and seed.

The decisive elements are the GLS weights and the effort spent in each separation. Additive weights increase the heights by 1.4% ($p < 0.001$). With a fifth of the time (24 s), the heights increase by 2.3% ($p < 0.001$) and most of the advantage over the published values is lost (the gap on Stoyan 2004 drops from $-2.9\%$ to $-0.1\%$): the method needs many moves, so their speed matters as much as their exactness. The number of passes of MinimizeOverlap also matters: with 100 passes the heights increase by 1.8% ($p < 0.001$), since separations that would succeed are abandoned, and with 2000 passes by 0.7% ($p = 0.008$), since the time is spent on hopeless ones. The compaction is needed: without it the heights increase by 0.4% ($p = 0.10$) and, in a separate run of all the instances, only 19 instead of 24 of the 27 optima of Section 4.4 are found, because the height of the best solution then decreases only in the discrete steps of the oscillation. The other elements have no significant effect ($p \geq 0.06$): the wall term ($\sigma = 0$: +0.5%), the tuned greedy criterion (greedy by volume: +0.3%), the step $\rho$, and the kick (no kick: +0.2%). On these instances the diversification thus comes from the restart itself, with reset weights and a new random order of the moves; the random move is needed only on small instances (Section 4.4).

### *4.3. Comparison with the literature*

Table 3 compares the method with the literature. On the instances that share the exact geometry of 3DNest (Stoyan 2005 Examples 2–3 and Merged1–5), the method obtains lower heights in six of seven cases; the exception is Stoyan 2005 Example 2 (19.14 versus 19.01). The improvement grows with the size of the instance: on Merged3–5 our best heights are 7–9% below the best known values, obtained by the voxel VNS with the finest resolution and four hours, and our mean heights after 120 s are lower than those best values. On Stoyan 2004, the method improves the exact reference by a wide margin, and on Example 2 it reaches the optimum.

On Stoyan 2004 Example 2 our height, 29, equals the lower bound $H_0$ given by the tallest piece, so it is optimal for the public data. Stoyan 2004 Example 1 (80 pieces) has no published reference and is not included; the instance called Experiment 1 by Lamas-Fernandez et al. (2023) is the one of Liu et al. (2015), where our mean height is 12% below the best voxel height

Table 2: Component analysis on the seven hardest instances (120 s, 10 seeds). Gap: mean gap of the height with respect to the best published value (Stoyan 2004 Examples 2–5 and Stoyan 2005 Examples 2–3); negative is better. Change: mean relative change of the height with respect to the proposed method over the 70 runs, i.e., over the seven instances including Stoyan 2004 Example 1, which has no published reference; positive means higher, i.e., worse ($p$: Wilcoxon signed-rank test).

| | Gap | | |
|---|---|---|---|
| Configuration | S2004 Ex2–5 | S2005 Ex2–3 | Change ($p$) |
| Proposed method | −2.9% | −2.0% | – |
| Without Compact | −2.6% | −1.6% | +0.4% (0.10) |
| $\sigma = 0$ (no wall term) | −2.3% | −1.7% | +0.5% (0.19) |
| Greedy by volume | −2.9% | −1.4% | +0.3% (0.22) |
| No kick (restarts only) | −2.6% | −2.1% | +0.2% (0.29) |
| Additive GLS weights | −1.4% | −0.8% | +1.4% (<0.001) |
| Time limit 24 s (default 120 s) | −0.1% | −1.2% | +2.3% (<0.001) |
| $\rho = 0.5\%$ (default 1%) | −2.8% | −2.1% | +0.1% (0.81) |
| $\rho = 2\%$ | −2.2% | −1.5% | +0.4% (0.06) |
| 100 passes (default 500) | −1.1% | −1.1% | +1.8% (<0.001) |
| 2000 passes | −2.0% | −1.2% | +0.7% (0.008) |

Table 3: Heights (lower is better). Exact: 3DNest (Egeblad et al., 2009) (best and mean of 10 runs of 600 s), optimum of Tollenaere et al. (2026) for Stoyan 2005 Example 1, and Stoyan et al. (2004) (St04). Voxels: VNS of Lamas-Fernandez et al. (2023) (best: standard resolution and 10 runs of 1 h for Stoyan, extra-fine resolution and 4 h for Merged; mean: standard resolution). Ours: mean (best) of 10 seeds, 120 s. *Optimal (equal to the lower bound $H_0$). †Value reported by Lamas-Fernandez et al. (2023); it is below the height of the tallest piece (29) in the public data. ‡HAPE3D (Liu et al., 2015), allowing 8 rotations. §Not exact: conservative convex decomposition (Section 4, Discussion); ours: 5 seeds of 20 min (15 min with a coarse decomposition and 5 min with a fine one); voxels: 10 runs of 1 h.

| | | Exact | | Voxels | | |
|---|---|---|---|---|---|---|
| Instance | $n$ | best | mean | best | mean | Ours |
| Stoyan 2005 Ex1 | 7 | 17.46 (opt.) | – | – | – | 17.47 (17.46) |
| Stoyan 2005 Ex2 | 12 | **19.01** | 19.83 | 21.23 | 21.70 | 19.16 (19.14) |
| Stoyan 2005 Ex3 | 25 | 28.83 | 29.82 | 29.76 | 31.53 | **27.45 (27.05)** |
| Stoyan 2004 Ex2 | 20 | 32 (St04) | – | 28.47† | 29.40 | **29.00 (29.00)*** |
| Stoyan 2004 Ex3 | 30 | 49 (St04) | – | 41.30 | 42.65 | **40.00 (39.13)** |
| Stoyan 2004 Ex4 | 40 | 63.21 (St04) | – | 54.83 | 59.52 | **53.32 (52.15)** |
| Stoyan 2004 Ex5 | 50 | 78.7 (St04) | – | 72.33 | 76.81 | **66.96 (65.13)** |
| Merged1 | 15 | 17.00 | 17.45 | 17.13 | 18.67 | **16.70 (16.56)** |
| Merged2 | 30 | 21.98 | 22.02 | 22.06 | 22.64 | **21.10 (21.01)** |
| Merged3 | 45 | 24.04 | 24.97 | 24.32 | 26.26 | **22.60 (22.01)** |
| Merged4 | 60 | 27.81 | 28.35 | 26.47 | 29.44 | **24.93 (24.12)** |
| Merged5 | 75 | 30.13 | 31.31 | 29.24 | 31.12 | **27.93 (27.07)** |
| Liu Exp. 1 | 36 | 31.2‡ | – | 34.27 | 35.37 | **30.22 (30.00)** |
| *Curved meshes (conservative decomposition)* | | | | | | |
| Chess§ | 32 | – | – | **131** | **135.1** | 148.4 (138.6) |

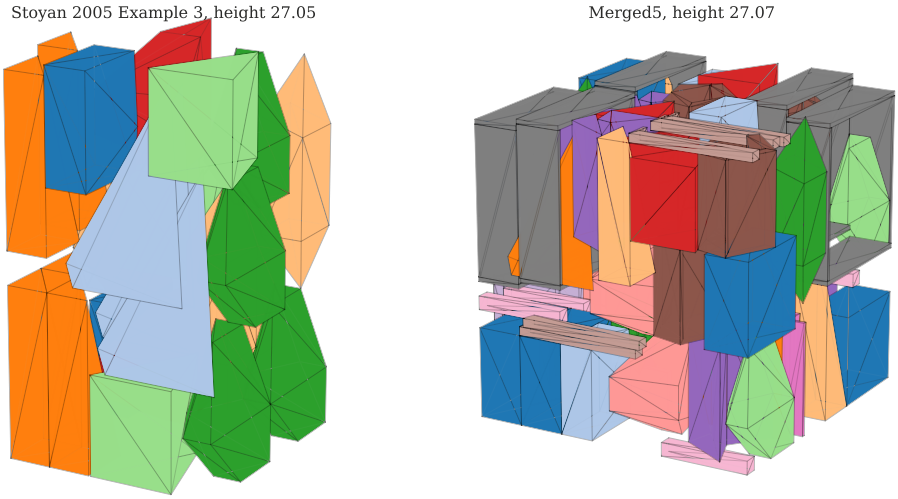



Figure 6: Best solutions of the final runs for Stoyan 2005 Example 3 and Merged5.

and also below that of HAPE3D, which allows rotations. For Stoyan 2005 Example 1, the utilization reported by Egeblad et al. (2009) converts to a height below the proven optimum with our data, so the instance seems to differ and we only compare with the optimum. Figure 6 shows two of the best solutions.

### *4.4. Comparison with exact models*

Tollenaere et al. (2026) solve four series of small instances (2–20 items of the convex pieces of Stoyan 2005 and Stoyan 2004, of diamonds and of tetrahedra) with mixed-integer models based on no-fit polyhedra, with a time limit of one hour per model; they prove optimality for 27 of the 66 instances and report upper and lower bounds for the others. Table 4 summarizes our results (10 seeds, 30 s per instance). The method finds 24 of the 27 proven optima in its best run, 12 of them in every run. Two of the others are missed by 0.03% (19.31 versus 19.30 and 24.94 versus 24.93, the 3-item instance of the Stoyan 2004 series and the 9-item instance of the Stoyan 2005 series), and the 10-item instance of the Stoyan 2005 series by 0.6% (26.20 versus 26.05). On the 39 open instances it improves the best upper bound in 32 cases, by 5% on average, and matches it in most of the others. The random move of the restarts is needed here: in an earlier version, whose restarts only reinserted the moved piece at its lowest position, the optimum of two 2-item

Table 4: Instances of Tollenaere et al. (2026) (10 seeds, 30 s). Optima: instances with proven optimum, and number of them found in the best run and in all runs. Open: instances without proven optimum, number in which our best height is below the best upper bound of the models, and mean gap of our mean height with respect to that bound.

| | | Optima | | | Open | |
|---|---|---|---|---|---|---|
| Series | $n$ | proven | best run | all runs | improved | gap |
| Diamonds | 9 | 4 | 4 | 2 | 5 of 5 | −4.5% |
| Stoyan 2004 | 19 | 9 | 8 | 3 | 9 of 10 | −4.8% |
| Stoyan 2005 | 19 | 9 | 7 | 5 | 9 of 10 | −4.7% |
| Tetrahedra | 19 | 5 | 5 | 2 | 9 of 14 | −1.7% |
| Total | 66 | 27 | 24 | 12 | 32 of 39 | |

instances was missed (by 17% and 28%), because the greedy places the larger piece on the floor, the optimum needs the opposite arrangement, and neither the GLS weights (a single pair) nor reinsertion can escape.

### *4.5. Time-matched comparison*

Hardware differs by about ten years: 3DNest ran on one core of a 2.16 GHz Intel Core Duo (2006), and the single-thread ratings of PassMark (PassMark Software, 2026) are 667 for that processor (Core Duo T2600) and 1624 for ours (Xeon E5-2620 v4), a factor of about 2.4 per core; this is rather an upper bound, since the rating uses the maximum turbo frequency and our runs share the processor. Figure 7 compares the mean utilization over time with that of 3DNest at 10, 60, 300 and 600 seconds, converted to our machine. Our first solution is available after about 0.3 seconds (the greedy construction), and at 10 seconds our mean utilization already exceeds the mean utilization of 3DNest at 600 seconds on all five instances (44.1, 43.1, 44.9, 43.8 and 43.1% versus 43.0, 42.9, 43.3, 42.0 and 41.0%). With the factor 2.4, our 10 seconds correspond to about 24 seconds on the machine of Egeblad et al. (2009), and their 600 seconds to about 250 seconds on ours, so the advantage holds at equal effective time by more than an order of magnitude.

### *4.6. Discussion*

The results support a conclusion that agrees with recent 2D work (Gardeyn et al., 2025): the quality of an overlap-minimization heuristic is driven by how many separation attempts fit in the time budget, and therefore by the

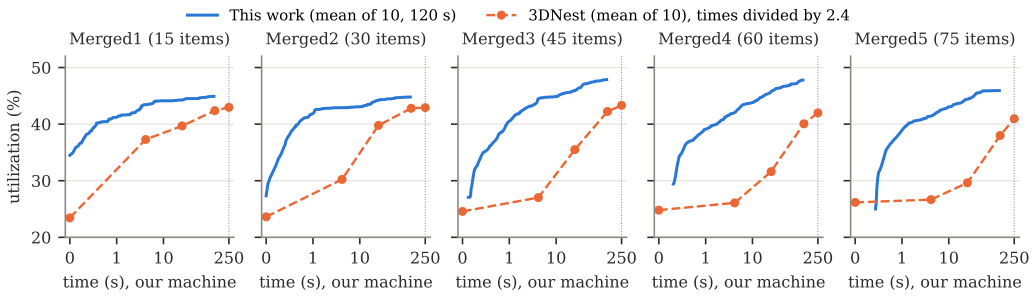



Figure 7: Mean utilization over time on the Merged instances (10 seeds), compared with 3DNest (Egeblad et al., 2009, Table 3) (10 runs). The times of 3DNest (10, 60, 300 and 600 s) are divided by 2.4, the ratio of the single-thread performance of the two processors, so that both curves are on the time scale of our machine; the dotted line marks the 600 s of 3DNest (250 s).

cost of evaluating and optimizing the overlap. The NFP-based penetration depth gives an exact, cheap and informative measure for unions of convex parts with few facets, which is the case of all the benchmark instances.

*Realistic meshes.* The cost of a move grows with the number of facets of the NFPs, so pieces with curved surfaces are the hard case. We tested the Chess instance of Lamas-Fernandez et al. (2023) (32 pieces, meshes of 700–2900 triangles). Since an exact convex decomposition of such meshes is impractical, we use a conservative one (the large convex hulls of the NFPs are computed with Qhull (Barber et al., 1996)): CoACD (Wei et al., 2022) cuts the mesh and takes the convex hull of each piece; the small gaps it leaves between parts are covered by scaling each part from its centroid, and containment is checked with interior sample points. With concavity threshold 0.05 (6–20 parts per piece) the union exceeds the volume of the piece by 4–12%, and with 0.2 (1–4 parts, reduced to at most 42 facet directions) by 38–78%. The solutions remain feasible for the real geometry. However, the NFPs between parts have hundreds of facets and a separation takes 20–35 s instead of a fraction of a second. In 20 minutes (5 seeds; Table 3) the best (mean) heights are 146.3 (151.2) with the fine decomposition, 149.9 (155.0) with the coarse one and 138.6 (148.4) with 15 minutes of coarse search followed by 5 minutes of refinement with the fine decomposition, against 131 (best of 10 runs of 1 h) and 135.1 (mean) for the voxel VNS of Lamas-Fernandez et al. (2023), whose cost does not depend on the complexity of the surface. Pseudo-blocks of two pieces did not help. The other realistic instance of Lamas-Fernandez

et al. (2023), Engine (97 pieces with meshes of up to 342,000 triangles), was not tested, for the same reason. Voxels and the proposed method are thus complementary: the latter is faster and exact on polyhedral pieces with few facets, the former scales better to curved meshes.

*Limitations.* Rotations are not considered. The method requires a convex decomposition of each piece; for the benchmark instances it is exact and small, while for curved meshes a conservative decomposition overestimates the pieces and, above all, makes the moves expensive. Reducing the cost of the NFPs of curved parts (e.g., hierarchical filtering with the hull of each piece, or conservative simplification of the NFPs) is left for future work.

## 5. Conclusions

We presented an overlap-minimization heuristic for 3D irregular strip packing that works with the exact geometry of polyhedra given as unions of convex parts. Its key element is an exact axis move based on penetration depths in no-fit polytopes, combined with multiplicative guided local search, strategic oscillation and restarts. This simple method outperforms the previous methods, with exact and voxel geometry, on the benchmarks of polyhedral pieces, and matches almost all the proven optima of the small instances. On curved meshes, which require conservative decompositions with many facets, it remains behind voxel-based search. Future work includes cheaper NFP computations for curved parts, rotations, larger instances from additive manufacturing, and lower bounds.

## Acknowledgments

This work was supported by ANID FONDECYT project 1260159.

## CRediT authorship contribution statement

**Ignacio Araya**: Conceptualization, Methodology, Software, Validation, Investigation, Writing – original draft, Writing – review & editing, Funding acquisition.

## Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

## Data availability

The source code, the convex decompositions of the pieces, the best solutions of Table 3 and the scripts that reproduce all the experiments are available at `https://github.com/rilianx/poly3d-packing`. The instances are available in the ESICUP repository (EURO Special Interest Group on Cutting and Packing, 2025).

## References


Agarwal, P.K., Guibas, L.J., Har-Peled, S., Rabinovitch, A., Sharir, M., 2000. Penetration depth of two convex polytopes in 3D. Nordic Journal of Computing 7, 227–240.

Akiba, T., Sano, S., Yanase, T., Ohta, T., Koyama, M., 2019. Optuna: A next-generation hyperparameter optimization framework, in: Proceedings of the 25th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining, pp. 2623–2631. doi:10.1145/3292500.3330701.

Barber, C.B., Dobkin, D.P., Huhdanpaa, H., 1996. The quickhull algorithm for convex hulls. ACM Transactions on Mathematical Software 22, 469–483.

Bennell, J.A., Oliveira, J.F., 2008. The geometry of nesting problems: A tutorial. European Journal of Operational Research 184, 397–415.

Cameron, S.A., Culley, R.K., 1986. Determining the minimum translational distance between two convex polyhedra, in: Proceedings of the 1986 IEEE International Conference on Robotics and Automation, pp. 591–596.

Egeblad, J., Nielsen, B.K., Brazil, M., 2009. Translational packing of arbitrary polytopes. Computational Geometry 42, 269–288.

Egeblad, J., Nielsen, B.K., Odgaard, A., 2007. Fast neighborhood search for two- and three-dimensional nesting problems. European Journal of Operational Research 183, 1249–1266.

Elkeran, A., 2013. A new approach for sheet nesting problem using guided cuckoo search and pairwise clustering. European Journal of Operational Research 231, 757–769.

EURO Special Interest Group on Cutting and Packing, 2025. ESICUP datasets. `https://github.com/ESICUP/datasets`.

Gardeyn, J., Vanden Berghe, G., Wauters, T., 2025. An open-source heuristic to reboot 2D nesting research. arXiv:2509.13329.

Gomes, A.M., Oliveira, J.F., 2006. Solving irregular strip packing problems by hybridising simulated annealing and linear programming. European Journal of Operational Research 171, 811–829.

Hertel, S., Mehlhorn, K., 1983. Fast triangulation of simple polygons, in: Foundations of Computation Theory, pp. 207–218.

Ikonen, I., Biles, W.E., Kumar, A., Wissel, J.C., Ragade, R.K., 1997. A genetic algorithm for packing three-dimensional non-convex objects having cavities and holes, in: Proceedings of the 7th International Conference on Genetic Algorithms, pp. 591–598.

Imamichi, T., Yagiura, M., Nagamochi, H., 2009. An iterated local search algorithm based on nonlinear programming for the irregular strip packing problem. Discrete Optimization 6, 345–361.

Jakobs, S., 1996. On genetic algorithms for the packing of polygons. European Journal of Operational Research 88, 165–181.

Lamas-Fernandez, C., Bennell, J.A., Martinez-Sykora, A., 2023. Voxel-based solution approaches to the three-dimensional irregular packing problem. Operations Research 71, 1298–1317.

Li, Z., Milenkovic, V., 1995. Compaction and separation algorithms for non-convex polygons and their applications. European Journal of Operational Research 84, 539–561.

Liu, X., Liu, J.m., Cao, A.x., Yao, Z.l., 2015. HAPE3D—a new constructive algorithm for the 3D irregular packing problem. Frontiers of Information Technology & Electronic Engineering 16, 380–390.

Lourenço, H.R., Martin, O.C., Stützle, T., 2003. Iterated local search, in: Glover, F., Kochenberger, G.A. (Eds.), Handbook of Metaheuristics. Kluwer Academic Publishers, pp. 320–353.

PassMark Software, 2026. CPU benchmarks: single thread performance. `https://www.cpubenchmark.net`. Accessed October 2026.

Romanova, T., Bennell, J., Stoyan, Y., Pankratov, A., 2018. Packing of concave polyhedra with continuous rotations using nonlinear optimisation. European Journal of Operational Research 268, 37–53.

Stoyan, Y.G., Gil, N.I., Pankratov, A., Scheithauer, G., 2004. Packing non-convex polytopes into a parallelepiped. Preprint MATH-NM-06-2004. Technische Universität Dresden.

Stoyan, Y.G., Gil, N.I., Scheithauer, G., Pankratov, A., Magdalina, I., 2005. Packing of convex polytopes into a parallelepiped. Optimization 54, 215–235.

Tollenaere, J., Martinez-Sykora, A., Wauters, T., 2026. Mixed-integer linear programming models for 3D irregular strip packing problems. European Journal of Operational Research 331, 365–380.

Umetani, S., Yagiura, M., Imahori, S., Imamichi, T., Nonobe, K., Ibaraki, T., 2009. Solving the irregular strip packing problem via guided local search for overlap minimization. International Transactions in Operational Research 16, 661–683.

Voudouris, C., Tsang, E., 1999. Guided local search and its application to the traveling salesman problem. European Journal of Operational Research 113, 469–499.

Wei, X., Liu, M., Ling, Z., Su, H., 2022. Approximate convex decomposition for 3D meshes with collision-aware concavity and tree search. ACM Transactions on Graphics 41, 42:1–42:18.